Stage 2: Literature Review

ML Diploma Graduation Project | Deliverable Template v1

Team name	Team 171
Project title	AI-Powered Predictive Maintenance for Industrial Equipment
Submission date	16th October, 2026
Version	v1

1. Research Questions

RQ1. Which models (classical machine learning or sequence models) give the best results on NASA C-MAPSS, both for classifying failure within the next 30 cycles and for predicting Remaining Useful Life (RUL), and which metrics and numbers do authors report?

RQ2. How do authors prepare C-MAPSS data (labels, train/test splits, scaling, class imbalance) so that their results are not inflated by data leakage?

RQ3. Which engineered features and tuning methods (rolling statistics, rate of change, PCA, grid or Bayesian search) are reported to help prediction?

RQ4. What do existing predictive-maintenance products offer, and what gap is left for an open, reproducible, deployed and monitored system?

2. Search Method

Sources searched	Web and scholarly search; results came from arXiv, Cambridge Core (AI EDAM), Elsevier/ScienceDirect, Springer, IEEE conference proceedings, PHM Society, MDPI, university repositories, the NASA Prognostics Center of Excellence data repository, and vendor documentation (Siemens, AWS, IBM).
Keywords used	predictive maintenance; C-MAPSS; FD001; turbofan engine degradation; remaining useful life; LSTM RUL; failure within 30 cycles classification; XGBoost; random forest predictive maintenance; time-series cross-validation; class imbalance SMOTE; drift monitoring.
Date range	2008 to 2026 (the C-MAPSS dataset appeared in 2008); most weight on 2016 to 2026.
Inclusion criteria	Uses C-MAPSS or similar run-to-failure sensor data, or is a product or dataset directly relevant to our pipeline; reports at least one number (RMSE, Score, precision, recall, F1, ROC-AUC or a dataset size); peer reviewed, a recognised conference, a well-documented preprint, or official documentation; in English. Excluded: blog posts and tutorials without an evaluation.
Sources found / selected	Approximately 25 candidates were screened. Eight research papers were selected for the summary table, alongside four dataset references and three product sources (15 references in the final bibliography).

3. Related Work Summary

#	Reference (authors, year, title)	Dataset	Method	Results	Limitations	Relevance to us
1	M. K. Tezcan and H. Kuscu (2026). Predictive Maintenance via Remaining Useful Life Estimation in Jet Engine Systems: A Comparative Analysis of Machine Learning Approaches Using the NASA CMAPSS Dataset.	NASA C-MAPSS FD001 (100 training engines, 100 test engines)	Linear Regression, Random Forest and Gradient Boosting using normalized sensor features and capped RUL labels.	FD001 test set: Random Forest RMSE 17.09, MAE 12.10, R^2 0.818; Gradient Boosting RMSE 17.42; Linear Regression RMSE 20.60.	RUL regression only; one FD001 split and no classification metrics. Results depend on preprocessing and label capping.	Provides a numerical tree-model baseline on FD001 and supports comparing interpretable, deployable models with sequence models.
2	S. Zheng, K. Ristovski, A. Farahat, C. Gupta (2017). Long short-term memory network for remaining useful life estimation.	C-MAPSS FD001–FD004	LSTM layers followed by fully connected layers on windows of sensor readings (RUL regression).	FD001: RMSE 16.14, NASA Score 338.	Regression only (no failure-alert metrics); higher error than later CNN work (RMSE 12.61).	LSTM reference for our sequence model; its RMSE of 16.14 is a reference point for our RUL model.
3	G. Sateesh Babu, P. Zhao, X.-L. Li (2016). Deep convolutional neural network based regression approach for estimation of remaining useful life.	C-MAPSS FD001–FD004	CNN on sliding windows of sensor data (RUL regression).	FD001: RMSE 18.45, NASA Score 1,290.	Largest error of the RUL papers we reviewed; regression only.	Early deep-learning baseline; its RMSE of 18.45 is the early reference level for RUL.
4	X. Li, Q. Ding, J.-Q. Sun (2018). Remaining useful life estimation in prognostics using deep convolution neural networks.	C-MAPSS FD001–FD004	Deep CNN over time windows of normalized sensor data (RUL regression).	FD001: RMSE 12.61, NASA Score 273.7.	Regression only; needs window-size tuning and is harder to interpret than tree models.	Best RUL error among our sources; upper reference for our RUL results.
5	M. Kimollo, X. Liu (2024). Aircraft engine remaining useful life prediction using machine learning.	C-MAPSS (100 engine units)	Binary classification of failure within a month (LSTM, random forest, k-NN and others), plus CNN-based RUL regression.	Classification: LSTM precision 0.95, recall 0.88, F1 0.91 (test set). RUL: CNN RMSE 14.02.	Short conference paper; one-month classification horizon is not the same as 30 operational cycles, so scores are not directly comparable.	Relevant two-task framing (classification and RUL), but its F1 = 0.91 is for a different prediction horizon.

#	Reference (authors, year, title)	Dataset	Method	Results	Limitations	Relevance to us
6	H. Özcan (2026). Predictive maintenance in aircraft engine maintenance using the C-MAPSS dataset: performance comparison and evaluation of machine learning classification algorithms.	C-MAPSS, 24 inputs (3 operating settings + 21 sensors), official train/test split	13 models (10 classical incl. RF, XGBoost, LightGBM; MLP, GRU, LSTM) for "failure within 30 cycles"; Min-Max scaling, SMOTE, grid/Bayesian search, SHAP, engine-level 5-fold CV.	LSTM: accuracy 0.981, macro-F1 0.92, ROC-AUC 0.96. GRU: ROC-AUC 0.97. LightGBM: accuracy 0.972, macro-F1 0.86, ROC-AUC 0.93. RF: accuracy 0.968, macro-F1 0.83, ROC-AUC 0.91.	Simulated data only, so scores may be optimistic for real fleets; high accuracy partly reflects the large healthy class; no deployment.	Closest to our core task (same label and metrics); main source of our target values.
7	T. Chen, C. Guestrin (2016). XGBoost: a scalable tree boosting system.	Several large public benchmarks (for example Higgs, Allstate, Criteo)	Regularized gradient-boosted decision trees with sparsity-aware split finding.	Higgs-1M classification: test ROC-AUC = 0.8304 for XGBoost versus 0.8302 for scikit-learn. This is a non-C-MAPSS benchmark and does not establish expected performance on our task.	Evaluated on non-C-MAPSS benchmarks; does not model sequences directly, so our task still needs temporal features and validation.	Why XGBoost is our main tree model; pairs with rolling and rate-of-change features.
8	A. Goel, S. Galchar, S. Kanu (2026). Remaining useful life estimation for turbofan engines: a comparative study of classical, CNN, and LSTM approaches (arXiv preprint).	C-MAPSS FD001 and FD003	Ridge and polynomial ridge, XGBoost, 1D CNN and LSTM under one identical preprocessing pipeline (RUL regression).	FD001 RMSE: LSTM 14.93, CNN 16.97. FD003 RMSE: LSTM 14.20, CNN 15.68, XGBoost 13.36.	Preprint, not peer reviewed as far as we know; regression only; FD002 and FD004 not covered.	Fair classical-versus-deep comparison, as we plan; tree models can match LSTM with engineered features.

4. Detailed Review of Key Works

Özcan (2026). The study asks the same question we do: will the engine fail within the next 30 cycles? It compares ten classical learners and three neural networks on C-MAPSS, using Min-Max scaling, SMOTE for the rare failure class, grid and Bayesian search, SHAP explanations and statistical tests. Sequence models came out on top (LSTM accuracy 0.981, macro-F1 0.92 and ROC-AUC 0.96; GRU ROC-AUC 0.97), with boosted trees close behind (LightGBM ROC-AUC 0.93, Random Forest 0.91). SMOTE helped some models and left

others unchanged, and the author warns that simulated data may make scores look better than they would be on real fleets. We will reuse the 30-cycle label, the metric set, and the rule of fitting scalers and resampling only on training folds. We will prioritize failure-class recall and F1 when choosing a model (which the study also reports) and go beyond an offline benchmark: our selected model will run behind an API with experiment tracking and monitoring.

Kimollo and Liu (2024). This paper treats predictive maintenance as two tasks: a yes/no question about failure within a month and regression of remaining operational cycles. This is similar to our core and secondary tasks, but a month is not necessarily 30 operational cycles. Its LSTM classifier reached precision 0.95, recall 0.88 and F1 0.91 on the test set, and its best CNN reached an RUL error of 14.02 RMSE. These results support evaluating classification and RUL together, but the classification score cannot be used as a directly comparable 30-cycle target. Our Stage 1 goals (failure-class recall ≥ 0.85 and F1 ≥ 0.80) reflect a practical target within our development time and computational limits, not a claim that our engine-level evaluation is stricter than every published study. We will reuse the two-task structure; we will differ by applying engine-level validation, comparing boosted trees with the LSTM, and delivering a serving layer.

Zheng et al. (2017). The authors fed windows of sensor readings into LSTM layers followed by fully connected layers to predict RUL and reported an RMSE of 16.14 and a NASA Score of 338 on FD001. It showed that a recurrent network can follow engine degradation over time. We will reuse the window-based input and RMSE plus Score as secondary RUL metrics, and we will use their RMSE as a reference point for our RUL model. We will differ by making the 30-cycle classification the main output and by testing whether tree models with rolling features can match the LSTM, as a recent comparison suggests (XGBoost RMSE 13.36 against LSTM 14.20 on FD003).

Li et al. (2018). This work applies a deep convolutional network over time windows of normalized sensor data and reached an RMSE of 12.61 and a Score of 273.7 on FD001, a clear improvement on the earlier CNN result of 18.45. It shows that careful normalization and window choice matter as much as the architecture. We will reuse the idea of normalizing sensors and working on windows, and we use 12.61 as the upper reference for our RUL model rather than as a goal. We will differ by not building a heavy CNN: our time goes into a fair comparison of three model families and into deployment.

5. Existing Solutions and Products

Product or tool	What it does	Gap your project addresses
Siemens Senseye Predictive Maintenance [13]	Cloud service that takes machine data such as vibration, current, torque and temperature and gives condition monitoring and failure insights to maintenance teams at large industrial sites.	Commercial, enterprise-scale and closed: models cannot be inspected or compared. We deliver an open, reproducible pipeline on a public benchmark with an explicit 30-cycle alert, optional RUL estimation if completed, and MLflow-tracked comparisons running locally in Docker.

Product or tool	What it does	Gap your project addresses
Amazon Lookout for Equipment [14]	Managed service that learned the normal behaviour of equipment from sensor data and flagged abnormal behaviour. AWS ended support on 7 October 2026.	Anomaly detection only, with no failure-within-N-cycles label or RUL; depended on a cloud vendor and is now discontinued. Our system predicts a defined failure window and is self-hosted.
IBM Maximo Application Suite (Maximo Predict) [15]	Enterprise asset-management suite whose Predict application uses AI and machine learning to forecast asset performance and maintenance needs.	Licensed suite that needs an asset-management system around it, and gives no view of how models were chosen. We focus on the model-to-service core and show the whole comparison and monitoring workflow openly.

6. Datasets Used in the Literature

Dataset	Source and link	Size	License	Suitable for us? Why
NASA C-MAPSS turbofan degradation (FD001–FD004)	NASA Ames Prognostics Data Repository [9]: nasa.gov/content/prognostics-center-of-excellence-data-set-repository	FD001: 100 training and 100 test engines, 20,631 training rows, 21 sensors + 3 operating settings.	Publicly downloadable from NASA; confirm dataset-specific reuse and redistribution terms before publishing raw files. Cite the NASA C-MAPSS repository [9].	Yes. Selected in our approved proposal (not mandated as FD001 in the main brief). FD001 has run-to-failure training histories and truncated test histories with separate RUL ground truth; it supports comparisons with studies [2]–[6], [8].
N-CMAPSS (run-to-failure under real flight conditions)	NASA repository (nasa.gov/content/prognostics-center-of-excellence-data-set-repository); Arias Chao et al. (2021)	Several sub-datasets; much larger flight-condition time series than C-MAPSS.	Free public download from NASA; check the repository terms.	Not now. More realistic, but far larger and more complex than our time allows; possible future work.
PRONOSTIA / FEMTO-ST bearing degradation	IEEE PHM 2012 challenge, hosted in the NASA repository (nasa.gov/content/prognostics-center-of-excellence-data-set-repository); Nectoux et al. (2012)	Run-to-failure experiments on 17 bearings (6 learning, 11 test); vibration and temperature.	Free public download; check the repository terms.	No. Raw high-frequency vibration needs signal processing and covers a different machine type.
AI4I 2020 Predictive Maintenance dataset	UCI Machine Learning Repository (archive.ics.uci.edu/dataset/601); Matzka	10,000 synthetic records, 14 columns.	CC BY 4.0.	No. One record per machine with no run-to-failure history, so it cannot support RUL, rolling features or sequence models.

7. Synthesis

(a) Baseline and best reported result. For RUL regression on FD001, the studies reviewed report RMSE 20.60 for Linear Regression and 17.09 for Random Forest in Tezcan and Kuscu [1], 18.45 for an early CNN [3], 16.14 for an LSTM [2], and 12.61 for a deep CNN [4]. These values are reference points rather than a controlled head-to-head comparison across papers. For the 30-cycle failure classification, which is our core task, a common strong baseline is a tree ensemble: in the widest comparison found (13 models) Random Forest reached macro-F1 0.83 and ROC-AUC 0.91, LightGBM macro-F1 0.86 and ROC-AUC 0.93, and the best results came from sequence models (LSTM macro-F1 0.92 and ROC-AUC 0.96; GRU ROC-AUC 0.97) [6]. A second study reported LSTM precision 0.95, recall 0.88 and F1 0.91 for failure within a month [5]; its horizon differs from our 30-cycle task. Macro-F1 should not be equated with F1 on the failure class.

(b) Metrics and methods everyone reports. RUL papers report RMSE and the NASA Score, which punishes late predictions more than early ones [2], [3], [4]. Classification papers report accuracy, precision, recall, F1 and ROC-AUC [5], [6]. On the feature and tuning side, Özcan [6] uses Min-Max scaling, SMOTE, grid and Bayesian search, SHAP, and engine-level 5-fold CV; Goel et al. [8] demonstrate the use of engineered sensor statistics, including mean, standard deviation, change, and slope, with classical models. Chen and Guestrin [7] establish XGBoost as an effective gradient-boosting algorithm. Together, these works motivate evaluating engineered temporal features with XGBoost, although their effectiveness on our 30-cycle classification task must be validated experimentally. PCA is mentioned only as an exploratory option, not as a reported improvement.

(c) How authors prepare data and avoid leakage (RQ2). Tezcan and Kuscu [1] use the official FD001 train/test split and fit Min-Max scaling on the training set only. Özcan [6] uses engine-level 5-fold cross-validation and fits scalers and SMOTE on training folds only. Kimollo and Liu [5] do not describe their split protocol in detail. The RUL papers [2], [3], [4] generally use the official train/test split but vary in whether they normalize before or after splitting, which makes their numbers hard to compare directly [5], [6].

(d) Pitfalls authors warn about. (i) Leakage: all cycles of one engine must stay in the same fold, and scalers and oversampling must be fitted on training folds only [6]. (ii) Class imbalance: failure cycles are rare, so accuracy misleads and SMOTE helps some models but not others [6]. (iii) Simulated data: scores on C-MAPSS may be optimistic compared with real fleets [6]. (iv) Inconsistent preprocessing between studies makes numbers hard to compare [6], [8].

(e) The gap our project fills. Among the sources reviewed, work usually stops at an offline benchmark: one best model on one split. We found no study among these sources that combines engine-level validation, a fair comparison of tree models and an LSTM, and a working system with an API, dashboard, experiment tracking and monitoring. Commercial tools cover monitoring but are closed and cannot be reproduced [13], [15]. Our project delivers that end-to-end, reproducible system on FD001.

Baseline and realistic target for our project. Baseline: a simple model (for example logistic regression) and the Random Forest and XGBoost models, which are also our fallback. Targets on held-out engines (Stage 1): failure-class recall ≥ 0.85, failure-class F1 ≥ 0.80 and ROC-AUC ≥ 0.90 for the 30-cycle classifier (O-01); for the secondary RUL model, RMSE ≤ 30 cycles with a stretch of MAE ≤ 25 (O-03). These deliberately conservative targets reflect the limited tuning time and the priority of delivering a reproducible end-to-end system. Published figures are reference points, not directly comparable guarantees: studies may use different preprocessing, label horizons, evaluation protocols and F1 definitions, and Özcan [6] also uses engine-level validation.

8. Impact on Our Design

Finding from the literature	Decision it drives in our project
Direct 30-cycle classification results include LSTM macro-F1 = 0.92 [6]; another two-task study reports LSTM F1 = 0.91 for a one-month horizon [5].	Keep the 30-cycle classifier as the core task (O-01) and RUL as secondary (O-03); compare RF, XGBoost and LSTM on the same split (O-02, T-08 to T-10).
Leakage inflates scores; folds must be engine-level and preprocessing fitted on training folds only [6].	Split by engine ID (T-05), fit scalers and any resampling inside training folds, and check every split in code review (Definition of Done, rule 3).
Accuracy is high mainly because healthy cycles dominate, and SMOTE helps only some models [6].	Report precision, recall, F1 (failure class) and ROC-AUC, not accuracy (O-01); start with class weights and threshold tuning, and test SMOTE as an experiment (T-08).
On the 30-cycle task, LightGBM ROC-AUC = 0.93 versus LSTM 0.96 [6]; separately, XGBoost RMSE 13.36 versus LSTM 14.20 on FD003 RUL [8].	Engineer rolling and rate-of-change features (T-07) and treat RF and XGBoost as full candidates; select the final model on validated evidence (T-10, T-13). Evaluate PCA as an exploratory feature-reduction method, not as a guaranteed improvement.
RUL references are RMSE 18.45 [3], 16.14 [2] and 12.61 [4], and the NASA Score punishes late predictions.	Report MAE and RMSE (optionally the Score) for the secondary RUL model; Stage 1 target RMSE ≤ 30 cycles, stretch MAE ≤ 25 (O-03, T-10).
C-MAPSS is simulated, so scores may be optimistic [6]; a managed vendor service was discontinued [14].	State the simulation limit in the final report; add basic drift checks and documented steps (O-05, T-19, T-20) and keep the stack open and self-hosted with Docker and MLflow (O-04).

9. References

[1] M. K. Tezcan and H. Kuscu, “Predictive maintenance via remaining useful life estimation in jet engine systems: A comparative analysis of machine learning approaches using the NASA CMAPSS dataset,” Engineering Proceedings, vol. 150, no. 1, p. 75, 2026. https://doi.org/10.3390/engproc2026150075

[2] S. Zheng, K. Ristovski, A. Farahat, and C. Gupta, “Long short-term memory network for remaining useful life estimation,” in Proc. IEEE Int. Conf. Prognostics and Health Management (ICPHM), Dallas, TX, USA, 2017, pp. 88–95. https://doi.org/10.1109/ICPHM.2017.7998311

[3] G. Sateesh Babu, P. Zhao, and X.-L. Li, “Deep convolutional neural network based regression approach for estimation of remaining useful life,” in Database Systems for Advanced Applications (DASFAA 2016), LNCS vol. 9642. Cham, Switzerland: Springer, 2016, pp. 214–228. https://personal.ntu.edu.sg/xlli/publication/RUL.pdf

[4] X. Li, Q. Ding, and J.-Q. Sun, “Remaining useful life estimation in prognostics using deep convolution neural networks,” Reliability Engineering & System Safety, vol. 172, pp. 1–11, 2018. https://doi.org/10.1016/j.ress.2017.11.021

[5] M. Kimollo and X. Liu, “Aircraft engine remaining useful life prediction using machine learning,” in Proc. Int. FLAIRS Conf., vol. 37, no. 1, 2024. https://doi.org/10.32473/flairs.37.1.135514

[6] H. Özcan, “Predictive maintenance in aircraft engine maintenance using the C-MAPSS dataset: Performance comparison and evaluation of machine learning classification algorithms,” AI EDAM, vol. 40, p. e4, 2026. https://doi.org/10.1017/S0890060426100249

[7] T. Chen and C. Guestrin, “XGBoost: A scalable tree boosting system,” in Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining, 2016, pp. 785–794. https://doi.org/10.1145/2939672.2939785

[8] A. Goel, S. Galchar, and S. Kanu, “Remaining useful life estimation for turbofan engines: A comparative study of classical, CNN, and LSTM approaches,” arXiv:2604.27234, Apr. 2026. https://arxiv.org/abs/2604.27234

[9] NASA Ames Prognostics Center of Excellence, “Turbofan engine degradation simulation data set (C-MAPSS),” NASA Ames Research Center, 2008. [Online]. Available: https://www.nasa.gov/content/prognostics-center-of-excellence-data-set-repository

[10] M. Arias Chao, C. Kulkarni, K. Goebel, and O. Fink, "Aircraft engine run-to-failure dataset under real flight conditions for prognostics and diagnostics," Data, vol. 6, no. 1, p. 5, 2021. https://doi.org/10.3390/data6010005

[11] P. Nectoux et al., "PRONOSTIA: An experimental platform for bearings accelerated degradation tests," in Proc. IEEE Int. Conf. Prognostics and Health Management (PHM'12), Denver, CO, USA, 2012.

[12] S. Matzka, "Explainable artificial intelligence for predictive maintenance applications," in Proc. 3rd Int. Conf. Artificial Intelligence for Industries (AI4I), 2020, pp. 69–74. Dataset: https://archive.ics.uci.edu/dataset/601/ai4i+2020+predictive+maintenance+dataset

[13] Siemens Digital Industries Software, “Senseye Predictive Maintenance: Product data sheet,” Aug. 2024. [Online]. Available: https://cache.industry.siemens.com/dl/files/319/109955319/att_1307608/v1/109955319_Senseye-product-sheet-V1.1_Aug24.pdf

[14] Amazon Web Services, “Amazon Lookout for Equipment,” product page and end-of-support notice. [Online]. Available: https://aws.amazon.com/lookout-for-equipment/ (accessed Oct. 15, 2026).

[15] IBM, “IBM Maximo Application Suite: Predictive maintenance,” product page. [Online]. Available: https://www.ibm.com/products/maximo/predictive-maintenance (accessed Oct. 15, 2026).